Project 2: Hidden Markov Models

Iván Luque García, Emma-Margaret Vaserik, Dalton Menhall

Model Description

State Space

The hidden state variable X_t takes one of 41 discrete values, one per phoneme in the ARPAbet set plus pau for silence. Each state is mapped to an integer index in {0, …, 40} using a fixed dictionary. The full set is: aa, ae, ah, ao, aw, ay, b, ch, d, dh, eh, er, ey, f, g, hh, ih, iy, jh, k, l, m, n, ng, ow, oy, p, r, s, sh, t, th, uh, uw, v, w, y, z, zh, pau, ax.

Observation Model

Each original frame produces an observation $O_t \in \mathbb{R}^{39}$. Emissions are modeled as a multivariate Gaussian per phoneme state:

$$P(O_t \mid X_t = i) = N(\mu_i, \Sigma_i)$$

where $\mu_i \in R^{39}$ is the mean vector and $\Sigma_i \in R^{39\times39}$ is the covariance matrix for phoneme i. The model assumes conditional independence of frames given the state sequence, which is the standard HMM emission assumption.

HMM Parameters

The model is defined by four components:

- π: initial state distribution, $\pi_i = P(X_1 = i)$, shape (41,)
- A: transition matrix, $a_{ij} = P(X_{t+1} = j \mid X_t = i)$, shape (41 x 41)
- μ_i: emission mean vector for phoneme i, shape (39,)
- Σ_i: emission covariance matrix for phoneme i, shape (39 x 39)

Implementation Details

Data Preparation and Label Alignment

Feature files (.npy) and label files (.lab) were matched by basename for each of the selected sets of validation, test and training data that were randomly grouped and picked from the original dataset. Each .lab file contains phoneme segments with end timestamps. To produce frame-level labels, we used np.searchsorted to map each segment end time to the corresponding frame index at a 10ms step size, then repeated each phoneme label for the number of frames it spans. The result is a label array of shape (T,) matching the feature array of shape (T, 39) for every utterance. Edge cases (sequence length mismatches) were handled by trimming or padding with the last label.

Supervised Parameter Estimation

π (Initial distribution): We count all instances of a phoneme as the first state and normalise by the number of test data utterances.

A (Transition matrix): We count all adjacent (X_t, X_{t+1}) pairs across all training utterances and normalize each row. Additive smoothing with $\alpha = 0.001$ is applied before normalization to avoid zero-probability transitions, which would cause $-\infty$ in log-space during inference, making a transition impossible for all future predictions.

μ, Σ (Emission parameters): All training frames assigned to phoneme i are collected and used to compute the sample mean and full covariance matrix. For phonemes with only one training frame, the covariance is set to λI. For phonemes with no training frames, both μ and Σ are set to zero and identity respectively. A regularization term λI ($\lambda = 0.00001$) is added to every covariance matrix to prevent singularity.

Numerical Stability

The forward algorithm uses row normalization at each timestep, dividing $\alpha(t)$ by its sum so the belief state always sums to 1. This prevents underflow over long sequences. If the sum is zero (no phoneme matches the observation), the previous belief state is carried forward as a fallback.

The Viterbi algorithm operates entirely in log-space. Emission probabilities are clamped to a minimum of 10^{-300} before taking the log to avoid log(0). Multiplications are replaced by additions throughout, and the transition scores are precomputed as log(A).

Numerical stability was checked in the following ways:

1. Forward normalisation was checked by seeing if rows encoding posterior probabilities sum to 1.
2. Synthetic extreme input was created to check that probabilities for any states could not reach absolute 0.
3. The predictions of all used inference algorithms were printed out for a few utterances to manually check that output is similar and reasonable.

Inference Algorithms

Forward Algorithm (Filtering)

The forward algorithm computes the normalized belief state $\alpha_i(t) = P(X_t = i \mid O_{1:t})$ at each frame. Initialization uses the emission probability of the first frame weighted by π. Each subsequent step predicts the next belief state by multiplying by A, then updates using the emission likelihood of the current observation. The argmax over states at each frame gives the frame-level phoneme prediction.

Viterbi Algorithm

Viterbi finds the single most probable state sequence $\hat{X}_{1:T}$ using dynamic programming in log-space. We maintain a score table δ(t, i) storing the log-probability of the best path to state i at time t, and a backpointer table psi(t, i) storing which previous state led to that best path. The recurrence is:

$$\delta_i(t) = max_j[\delta_j(t-1) + log\, a_{ji}] + log\, P(O_t \mid X_t = i)$$
$$psi(t,i) = arg\, max_j[\delta_j(t-1) + log\, a_{ji}]$$

After processing all frames, we backtrack from the highest-scoring final state to recover the full path. Unlike filtering, Viterbi enforces global path consistency, which typically produces smoother and more accurate phoneme sequences.

Smoothing Algorithm (Additional Inference Algorithm)

We decided to use as an addition, the smoothing algorithm which computes $P(X_t \mid O_{1:T})$ using both past and future observations. It runs a forward pass (identical to filtering) and a backward pass initialized at $log\, \beta_i(T) = 0$ and recursed as:

$$log\, \beta_i(t) = logsumexp_j[log\, a_{ij} + log\, P(O_{t+1} \mid X_{t+1} = j) + log\, \beta_j(t+1)]$$

The two passes are combined to give the smoothed posterior, and the argmax over states gives the per-frame prediction:

$$log\, \gamma_i(t) = log\, \alpha_i(t) + log\, \beta_i(t) \qquad X_t^* = arg\, max_i\, \gamma_i(t)$$

Both passes operate in log-space using logsumexp for numerical stability.

We wanted to try smoothing to see the difference between Viterbi and Smoothing's performance.

Discussion of Results

In the following section we will showcase the different results obtained from the project and discuss them.

Filtering vs. Viterbi vs. Smoothing

Frame accuracy summary

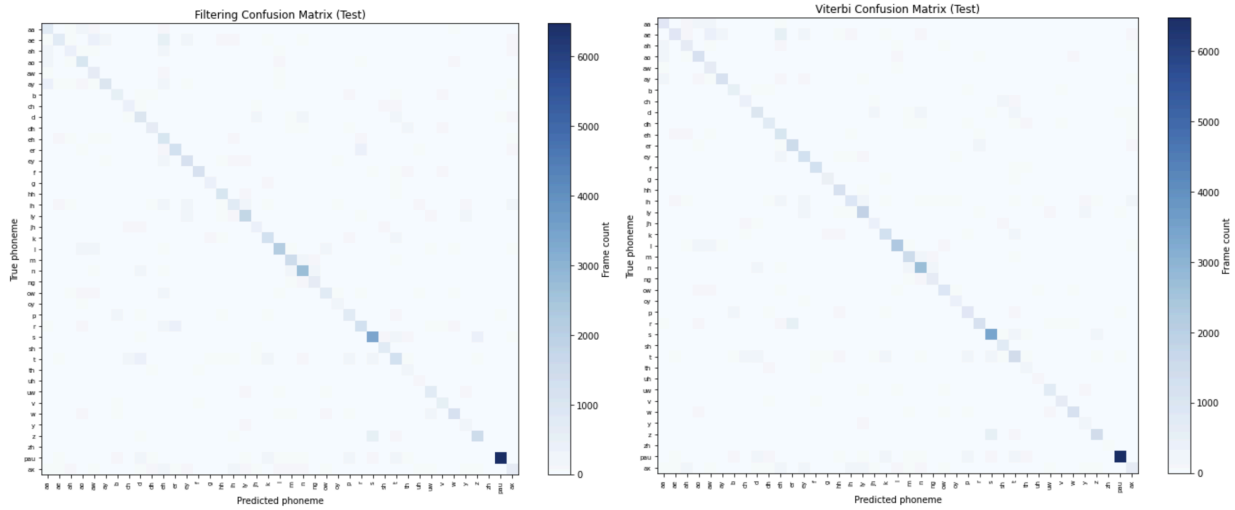
method	split	utt	frames	frame_acc	utt/s	time(s)
Filtering	train	792	237301	66.47%	13.35	59.3
Filtering	val	113	32323	64.95%	13.63	8.3
Filtering	test	227	69012	63.64%	13.21	17.2
Viterbi	train	792	237301	69.50%	45.37	17.5
Viterbi	val	113	32323	67.47%	39.65	2.9
Viterbi	test	227	69012	66.83%	45.17	5.0
Smoothing	train	792	237301	69.49%	12.01	65.9
Smoothing	val	113	32323	67.43%	11.51	9.8
Smoothing	test	227	69012	66.83%	12.33	18.4

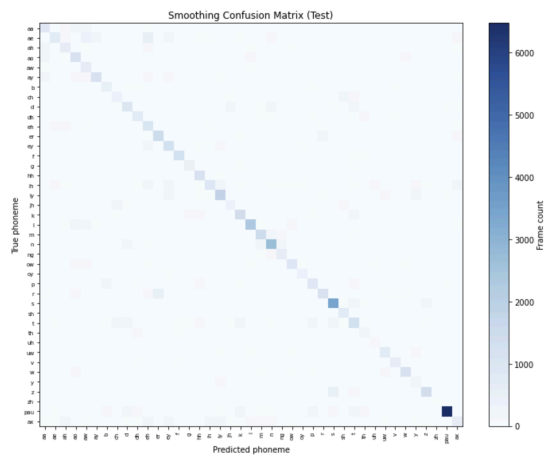
From testing the different Inference Algorithms on all the data sets, we found Viterbi performs better both on test and train data when compared to filtering. This is because Viterbi takes the most probable state sequence, whereas filtering looks at the individual states. Considering that our model analyses utterances, the sequence in which phonemes are spoken matters, making the analysis of the whole sequence of phonemes important in making better predictions, as viterbi also does. Additionally, the smoothing and viterbi perform very similarly, though viterbi performs slightly better. This is due to the same reason that viterbi considers whole sequences rather than individual states.

Train vs. Test vs. Validity Accuracy

We can observe the training accuracy is higher, mainly because it is tested on more data and that makes the accuracy increase considerably. But also the difference is not that excessive suggesting that the model is well fitted and there is no overfitting.

Confusion Matrices





From the confusion matrices of the different inference algorithms we observe that the most common phoneme is “pau”, following “s”, “n” and “i”. That means that these phonemes are the most correctly predicted phonemes. It makes sense that if we look outside the principal diagonal all the areas are more faded, since the model does not tend to misspredict the phonemes that often.

Per-Phoneme Precision and Recall

The best performing 5 phonemes were sh, pau, s, f, m when ordering the phonemes according to their recall probability, i.e., what percentage of instances of the spoken phonemes did the model actually correctly predict. This makes sense for pau, s, f, m as they were also one of the most frequently occurring phonemes. However, the presence of sh in this ranking could be due to the fact that it is very different from other phonemes, making it easier to make inferences based on evidence.

Per-phoneme precision/recall - Filtering

phoneme	precision	recall	support
sh	0.707	0.885	819
pau	0.963	0.855	7576
s	0.804	0.798	4159
n	0.733	0.780	3449
f	0.859	0.779	1540
m	0.684	0.755	1888
ng	0.612	0.749	824
k	0.616	0.740	1757
hh	0.698	0.714	1461
l	0.776	0.687	3079
z	0.684	0.682	2094
eh	0.342	0.674	1565
iy	0.701	0.666	2660
w	0.733	0.665	1653
aw	0.394	0.663	882
ey	0.548	0.655	1719
oy	0.580	0.651	444
uw	0.572	0.644	1030
ao	0.511	0.643	1617
v	0.609	0.642	862
p	0.531	0.635	1263
er	0.684	0.632	2071
g	0.587	0.600	655
b	0.531	0.589	820
y	0.364	0.586	435
ow	0.653	0.576	1353
dh	0.629	0.555	1156
r	0.620	0.554	2251
aa	0.450	0.550	1399
ch	0.445	0.544	586
th	0.301	0.508	356
d	0.456	0.498	1998
jh	0.474	0.497	646
ay	0.712	0.496	1906
t	0.503	0.458	2827
uh	0.203	0.403	149
ih	0.504	0.357	2231
ah	0.417	0.337	1120
zh	0.392	0.333	60
ae	0.655	0.314	2436
ax	0.405	0.291	2216

Per-phoneme precision/recall - Viterbi

phoneme	precision	recall	support
sh	0.683	0.910	819
pau	0.960	0.854	7576
f	0.856	0.834	1540
s	0.802	0.831	4159
m	0.716	0.784	1888
n	0.749	0.781	3449
hh	0.695	0.765	1461
ng	0.599	0.757	824
k	0.628	0.747	1757
l	0.808	0.741	3079
aw	0.422	0.736	882
ey	0.588	0.734	1719
w	0.777	0.719	1653
uw	0.613	0.711	1030
oy	0.658	0.709	444
er	0.662	0.706	2071
g	0.651	0.693	655
y	0.418	0.690	435
iy	0.729	0.687	2660
z	0.727	0.671	2094
ao	0.558	0.668	1617
ow	0.679	0.666	1353
b	0.585	0.661	820
eh	0.400	0.660	1565
p	0.555	0.655	1263
dh	0.632	0.654	1156
v	0.697	0.648	862
ay	0.761	0.622	1906
aa	0.550	0.590	1399
jh	0.504	0.556	646
ch	0.444	0.553	586
th	0.342	0.528	356
r	0.664	0.520	2251
uh	0.294	0.503	149
zh	0.769	0.500	60
ah	0.472	0.497	1120
d	0.505	0.494	1998
t	0.522	0.469	2827
ih	0.586	0.391	2231
ae	0.684	0.335	2436
ax	0.440	0.291	2216

Per-phoneme precision/recall - Smoothing

phoneme	precision	recall	support
sh	0.687	0.910	819
pau	0.961	0.854	7576
s	0.801	0.832	4159
f	0.854	0.831	1540
m	0.712	0.788	1888
n	0.751	0.782	3449
hh	0.696	0.764	1461
ng	0.605	0.758	824
k	0.629	0.750	1757
l	0.808	0.743	3079
aw	0.419	0.738	882
ey	0.586	0.736	1719
w	0.781	0.725	1653
uw	0.618	0.718	1030
oy	0.673	0.709	444
er	0.656	0.696	2071
g	0.661	0.692	655
y	0.411	0.687	435
iy	0.727	0.683	2660
z	0.726	0.670	2094
ao	0.559	0.666	1617
ow	0.686	0.666	1353
b	0.588	0.665	820
eh	0.400	0.663	1565
dh	0.634	0.662	1156
p	0.561	0.658	1263
v	0.693	0.652	862
ay	0.756	0.620	1906
aa	0.558	0.601	1399
jh	0.506	0.559	646
ch	0.432	0.543	586
th	0.333	0.522	356
r	0.656	0.515	2251
uh	0.285	0.503	149
ah	0.478	0.500	1120
zh	0.769	0.500	60
d	0.501	0.490	1998
t	0.519	0.464	2827
ih	0.589	0.390	2231
ae	0.686	0.330	2436
ax	0.441	0.291	2216

Example Utterances

In the example below we can observe the 3 first utterances, filtering performs clearly more poorly than the other two inference algorithms. That can be justified because the other two algorithms take into consideration future observations.

```
Utterance index: 0
Ground truth: pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau ao ao ao ao ao ao ao ao ao ao ao ao ao ao ao ao ao th th th th th
Filtering:    pau pau pau k k k k k pau pau pau b k t t t t t b p p ay ay ao ao ao ao ao ao ao ao ao ao ah ah ah th f f f
Viterbi:      pau pau pau k k k pau pau pau pau pau t t t t t t p p p p ao ao ao ao ao ao ao ao ao ao ao ah ah ah f f f f f
Smoothing:    pau pau pau k k k pau pau pau pau pau t t t t t t p p p p ao ao ao ao ao ao ao ao ao ao ao ah ah ah f f f f f
Frame accuracy -> Filtering: 61.68%, Viterbi: 67.37%, Smoothing: 67.96%

Utterance index: 1
Ground truth: pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau l l l l l l l l l l ao ao ao ao ao ao ao ao
Filtering:    pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau k k d d t t t y w w w l l l l l l l l l l ao ao ao ao ao
Viterbi:      pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau k k t t t t y y ax ax l l l l l l l l l l ao ao ao ao ao ao
Smoothing:    pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau k k t t t t y y ax ax l l l l l l l l l l ao ao ao ao ao ao
Frame accuracy -> Filtering: 62.00%, Viterbi: 64.80%, Smoothing: 63.60%

Utterance index: 2
Ground truth: pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau g g g g g g ae ae ae ae ae ae ae ae ae ae ae ae ae ae ae ae
Filtering:    pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau t t t y y y y eh eh eh eh eh ae ae ae ae ae ae ae ae
Viterbi:      pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau t t t t y y y ae ae ae ae ae ae ae ae ae ae ae ae ae ae
Smoothing:    pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau pau t t t t y y y ae ae ae ae ae ae ae ae ae ae ae ae ae ae
Frame accuracy -> Filtering: 63.00%, Viterbi: 72.69%, Smoothing: 68.28%
```

General Assessment

The general performance of the model is okay, considering that the best seen performance on the validation set is 67.47 % with Viterbui, meaning that over half of the phonemes could be predicted correctly. Additionally, as the performance on the validation set doesn't differ vastly from that of the training set, there was not overfitting to training data.

Tried optimisation (did not work)

The idea we had for optimising the model was to include how long a phoneme has lasted as part of the state representation. This was done to facilitate switching when a phoneme has lasted longer, as the original model was inclined to keep predicting the same phoneme as the next state as a general rule. Therefore, each state was represented as a tuple of (p, d) with p indicating phoneme and d indicating duration. If a new phoneme was spoken, the duration was encoded as d = 0 and for each consecutive timeframe where the same phoneme was spoken, the duration was incremented by d +=1. To keep the state space to a finite size, the duration was capped at max_duration, where all durations above the threshold were not counted incrementally but encoded as the same (maximum) duration. While the initial distribution was calculated similarly to the original approach, the transition matrix was calculated in the following way by calculating two different types of probabilities:

1) Probability of switching from one phoneme to another at each state, for each possible next phoneme. The probability of switching to a specific phoneme is normalised by the number of non-sparse switching transitions.
2) The probability of persisting in the same phoneme. This probability is integrated as the probability of transitioning to state (p,d+1) or (p,max_duration) and normalised by the number of non-sparse persistence transitions.

Furthermore, the regularisation (for eliminating 0 probability in transition matrix) was only applied to state that were possible under the model assumptions: 1) switch transitions only to state where d = 0; 2) persist transitions only to state where duration of next state is d+1 or equal to max_duration. This was done to keep all the 'impossible' state transitions at a probability of 0. The hyperparameters that could've been tuned (but we did not as we lacked the time resources)

are: 1) max_duration, 2) lambda (regularisation for covariance matrix), 3) alpha (regularisation for transition matrix) and 4) possibly an additional term that could've been implemented and would've weighted the transition probabilities to, so that the model would rely more (or less) on the transition model.